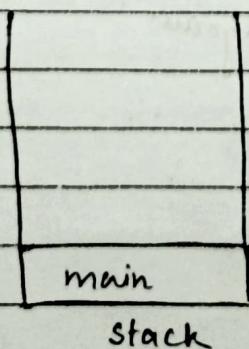


RECURSION

- function calling another function.
- How do function call works

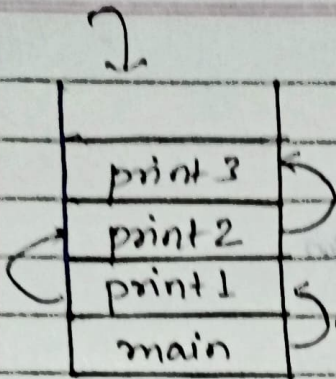
```
public class main {
    public static main (String [] args) {
        print1 (1);
    }
    static void print1 (int n) {
        sout (n);
        print2 (2);
    }
    static void print2 (int n) {
        sout (n);
        print3 (3);
    }
    static void print3 (int n) {
        sout (n);
        print3 (3)
    }
}
```



1. main function will be called 1st & stored in stack memory
2. As main function has print1 function it will call print1.
3. After calling it will wait for print1 function to complete then move forward.

Output

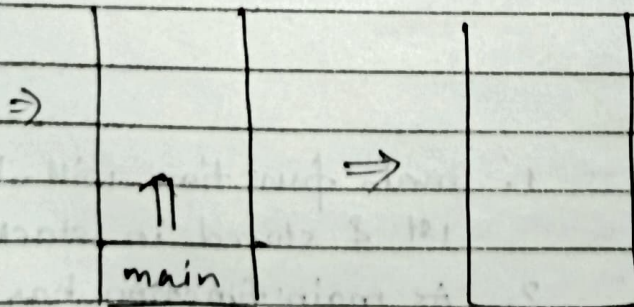
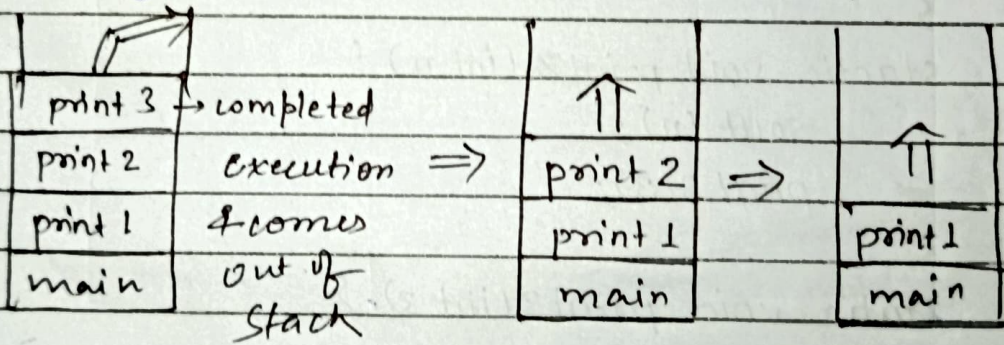
3
2
1



• then after print 1 prints '1' it will call print 2 function.

• it keep doing like this till function print 3 which don't have any function to call. So it will be executed completely.

Once print 3 is executed completely it will say print 2 & the flow of program returns to print 2 then print 1 then main & program is executed.



program is executed.

• Notes:-

- while the function is executing it will ~~be~~ ^{remain} in stack memory.

= ~~when a function finishes executing it is removed~~

- when a function completed/finishes executing it is removed from stack memory & flow of program is restored to where the function is called.

- Working of Recursive function:-

- it starts from main function.
- main function is called & stay in stack memory.
- ~~memory~~ main function calls print function.
- it check the base condition ~~true~~ ^{false} & then print & call it self with new parameters.
- it continues till base condition if true, where the function stop calling itself. & recursion stops.
- then if while function gets completely executed it ~~comes out of the loop~~ is removed from stack memory.

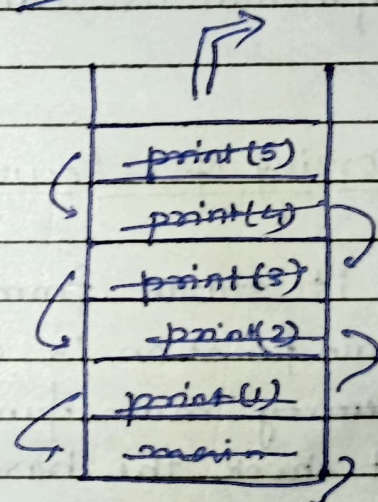
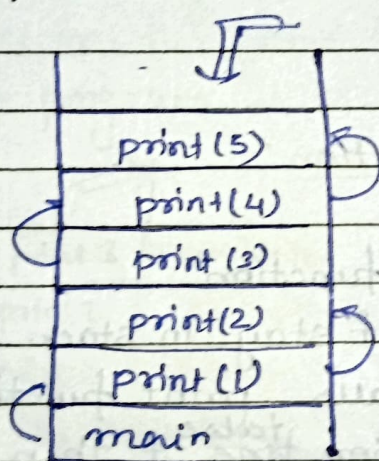
Code // .

```

=> public class main {
    public static void main (String[] args) {
        print (1);
    }
    static void print (int num) {
        if (num > 5) return; } Base Case
        System.out.println (num); }
        print (num+1); } Recursive case
    }
}

```

fail recursion



empty stack
program executed.

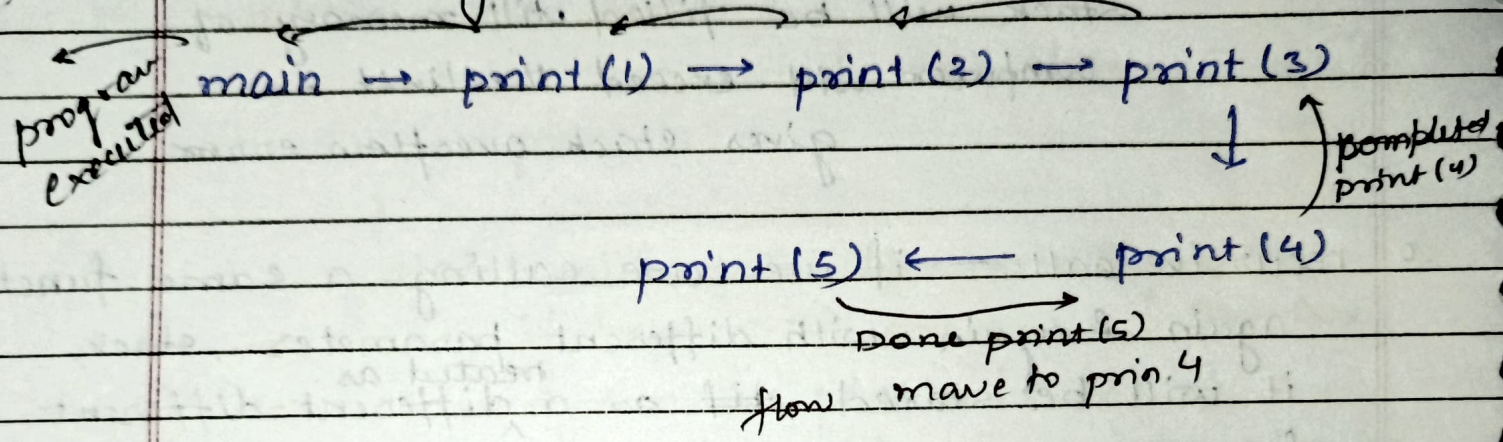
- Base Condition :- Condition that stops the recursion . prevent infinite calls
- Recursive Case :- Function calls it self with a simpler, or smaller input.

- No base condition:- function calls will keep happening stack will be filled till memory of computer will exceed the limit $\rightarrow$ gives stack overflow error.
- recursive call:- if we are calling a same function again & again with different parameters, stack it will be stored ~~diff~~ ^{created as} as a different-different functions.

* Why Recursion *

- $\Rightarrow$ What is Recursion?
 - function calling itself.
 - Recursion is a programming technique where a function call itself to solve a smaller instance of the same problem.
- $\Rightarrow$ Why Recursion?
 - It helps us in solving bigger / complex problem in a small instance simple way by solving a small instance of problem repeatedly.
 - we can convert recursion sol into iteration (loops) & vice versa.
 - Space complexity is not constant because of recursive calls.

Visualising recursion (Recursion Tree)



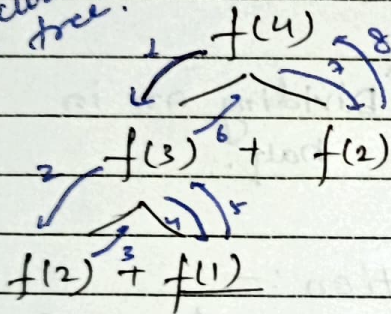
fibonacci Number $\Rightarrow$ 0, 1, 1, 2, 3, 5, 8, 13, ...

~~find~~ • find n^{th} fibonacci Number

~~f(5)~~ ~~f(4)~~

$$f(n) = f(n-1) + f(n-2) \Rightarrow \text{recurrence relation.}$$

Recursive tree.



Base Case.

$$f(1) = 0$$

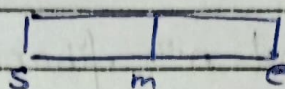
$$f(2) = 1$$

• Tail Recursion :- Type of Recursion where recursive call is last thing executed by the function.

• How to approach to Recursion Question :-

- Identify if you can break problem into small problems.
- Write the recurrence relation needed.
- Draw recursive tree
- About the tree :- See the flow of function.
 - Identify & focus on left & right tree calls.
- see how values are returned

(*) Binary Search : with recursion.

$\Rightarrow$ arr =  $\Rightarrow$ N element

comparing middle :- $O(1)$

dividing array into half = $N/2$

$$F(N) = O(1) + F(N/2)$$

/
comparision

\
Dividing arr in half.

Types of recurrence relation :-
 $\Rightarrow$ Linear Recurrence Relation $\rightarrow$ fibonacci

Divide & conquer Relation $\rightarrow$ Binary Search

Variables :-

(1) Arguments

(2) Return Type

(3) Body of function.